

STUDY STEP 15 OF 18

Graphs: Traversal, Ordering, Paths, and Union-Find

Modeling, BFS, DFS, Topological Sort, Shortest Paths, and Connectivity

FOUNDING AUTHOR

Vinay Reddy Kalluri

EDITOR-IN-CHIEF | CHIEF AUDITOR

Translate domains into vertices and edges, then choose traversal, ordering, path, or connectivity algorithms from graph constraints.

MODELING | BFS | DFS | TOPOLOGICAL SORT | DIJKSTRA | UNION-FIND

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Release 2026-07-25

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

60-second entry rule: If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to **Study Step 14 – Heaps and Priority Queues**. If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

Graph problems become manageable once the model and edge semantics are explicit. This volume emphasizes algorithm selection, state, overflow, and correctness.

Readiness map

Decision	Evidence
START HERE	Entities are connected by arbitrary relationships; Dependencies require an order or cycle test; Reachability, components, or shortest paths are requested; Grid cells can be modeled as graph vertices.
PREPARE FIRST	Study Steps 01-14; Queues, recursion, heaps, and hashing.
MOVE ON WHEN	Model graph state deliberately; Implement BFS, DFS, topological sort, Dijkstra, and union-find; Select a shortest-path algorithm from weight constraints; Explain complexity using V and E.

Choose the route that fits today

- 1 **Learning:** read in order, type the representative Java examples, and complete each dry run.
- 2 **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

Study Path Position

14 - Heaps and Priority Queues	YOU ARE HERE 15 - Graphs	16 - Greedy Algorithms
--------------------------------	------------------------------------	------------------------

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION	
Start Here - Your Route Through This Volume	2
Part I - Core Learning	4
Chapter 1 - SDE-2 Graph Modeling and Algorithm Patterns	4
Part II - Practice and Handoff	29
Practice Ladder and Next Steps	29

Part I - Core Learning**SDE-2 CORE CHAPTER**

Chapter 1 - SDE-2 Graph Modeling and Algorithm Patterns

Why graph interviews begin before traversal

A graph is a model of entities and relationships. The same business data can become directed or undirected, weighted or unweighted, static or streaming, simple or multi-edge. That choice determines which algorithm is correct. Running Dijkstra on a graph that contains negative edges is not a small implementation bug; it is a modeling and precondition failure.

At SDE-2 level, begin by defining vertices, edges, direction, weight meaning, duplicate/self-loop policy, and scale. Then choose a representation and algorithm. A complete answer states the invariant that makes a traversal, relaxation, or cut decision safe, reconstructs requested outputs, and identifies the boundary where an in-memory textbook solution no longer fits.

Learning objectives

After completing this chapter, you should be able to:

- translate a domain problem into a graph with explicit direction and weight semantics;
- compare adjacency lists, matrices, edge lists, and implicit graphs;
- implement BFS, DFS, connected components, and grid BFS;
- detect undirected and directed cycles and test bipartiteness;
- derive topological sorting from indegree or DFS finishing state;
- choose among BFS, Dijkstra, Bellman-Ford, DAG relaxation, and Floyd-Warshall;
- run Dijkstra with stale-entry handling and reconstruct a shortest path;
- derive minimum spanning trees with Kruskal/DSU and explain Prim's alternative;
- identify SCC and bridge algorithms as advanced graph-decomposition tools; and
- discuss stack depth, memory, disconnected data, large graphs, and distributed boundaries.

Model the graph explicitly

Before selecting an algorithm, answer:

- 1 **Vertex:** what is one node, and how is it identified?
- 2 **Edge:** what relation creates an edge?
- 3 **Direction:** does $u \rightarrow v$ imply $v \rightarrow u$?
- 4 **Weight:** cost, time, distance, probability, capacity, or something else?
- 5 **Multiplicity:** can parallel edges or self-loops exist?
- 6 **Dynamics:** is the graph fixed during the query?
- 7 **Output:** reachability, one path, all distances, an ordering, components, or a spanning structure?
- 8 **Scale:** how many vertices and edges, and can they fit in memory?

Examples:

- Build prerequisites are directed edges from prerequisite to dependent task.
- Mutual friendship is usually undirected; following is directed.
- A grid is an implicit graph whose open cells are vertices and legal moves are edges.
- Currency exchange with multiplicative rates needs a transformed model, not an unexamined "weighted shortest path."
- Network capacity is not a shortest-path weight; it may require max flow.

If a problem says "least number of transfers," every transition has unit cost and BFS is the first choice. If it says "least travel time" with nonnegative times, use Dijkstra. Meaning precedes syntax.

Representations and cost

Let V be vertices and E edges.

Representation	Space	Enumerate neighbors	Test one edge	Best fit
adjacency list	$O(V + E)$	$O(\text{degree}(v))$	usually $O(\text{degree}(v))$	sparse graphs and traversal
adjacency matrix	$O(V^2)$	$O(V)$	$O(1)$	dense small graphs
edge list	$O(E)$	$O(E)$ without index	$O(E)$	Kruskal, Bellman-Ford, interchange
implicit neighbors	domain state only	cost to generate	domain-specific	grids, puzzles, generated states

For weighted Java graphs, an adjacency list may be `List<List<Edge>>`. For dense integer IDs, `int[][]` or primitive arrays reduce boxing. For external string IDs, map them to compact integer indexes and retain a reverse mapping for output.

An undirected edge must appear in both adjacency lists unless the traversal generates both directions. Parallel edges may be valid; deduplicating them can change path or capacity semantics. Self-loops immediately form directed cycles and undirected cycles under many definitions, so state the chosen graph model.

WHY IT WORKS | Universal traversal invariant

BFS and DFS differ in frontier order, not reachability correctness. Both maintain:

- discovered vertices have been scheduled at most once;
- every scheduled vertex is reachable from the start through discovered edges; and
- when a vertex is processed, all its outgoing edges are examined.

Mark a vertex discovered when adding it to the frontier, not when removing it. Marking late permits many duplicate enqueues on converging edges and can destroy linear bounds.

Traversal time is $O(V + E)$ for an adjacency list because each vertex is scheduled once and every adjacency entry is examined once. For an undirected list, each logical edge appears twice, which remains $O(E)$.

Pattern 1: breadth-first search

BFS uses a FIFO queue. It processes vertices by nondecreasing number of edges from the source. In an unweighted graph, the first discovery of a vertex therefore gives a shortest edge-count path.

Invariant when dequeuing a vertex at level d : every vertex at a smaller distance has been processed, and no undiscovered path with fewer than $d + 1$ edges remains. Assign a neighbor distance $d + 1$ when it is first discovered and optionally store its parent.

TRACE IT | Dry run

For adjacency $0: [1, 2], 1: [0, 3], 2: [0, 3], 3: [1, 2, 4], 4: [3]$, BFS from 0 visits $0, 1, 2, 3, 4$. Levels are $0, 1, 1, 2, 3$. Either $0-1-3-4$ or $0-2-3-4$ is a valid shortest path, depending on neighbor order.

BFS memory can be $O(V)$ and its frontier may be very wide. Bidirectional BFS can reduce explored depth for one source-target query in an unweighted graph when reverse neighbors are available and branching is high.

Pattern 2: depth-first search

DFS explores one branch before backtracking. It supports reachability, components, cycle state, topological finishing order, and low-link decompositions.

Recursive DFS mirrors the proof but can overflow the Java stack on a long chain. An explicit `ArrayDeque<Integer>` is safer for untrusted or large graphs. Pushing neighbors in reverse gives a deterministic left-neighbor preference, although mark-on-push behavior around cross edges can still differ from recursive visitation order.

Invariant: the stack contains discovered work whose reachable outgoing edges are not all processed. Popping a vertex in the simple iterative traversal commits it to output; every newly pushed neighbor has a discovered path through the current vertex.

DFS remains $O(V + E)$ and may hold $O(V)$ frontier/visited state. Do not claim $O(\text{depth})$ total space while ignoring the visited set required for a general cyclic graph.

Pattern 3: connected components

For an undirected graph, scan every vertex. When a vertex is unseen, increment the component count and traverse from it. That traversal marks exactly one connected component.

Invariant before scan index v : every earlier vertex belongs to a fully marked component. An unseen v cannot belong to any already traversed component, or reachability would have marked it. It therefore starts a new component.

This handles isolated vertices, which count as size-one components. Time is $O(V + E)$, space $O(V)$. A disjoint-set union structure is an alternative when edges arrive incrementally and queries ask whether vertices are currently connected.

Pattern 4: grid as an implicit graph

For four-direction movement, each open cell has up to four neighbors. BFS finds the fewest steps when every move costs one. Validate rectangularity, coordinates, and blocked endpoints.

Use a distance matrix initialized to -1 . Mark the start distance zero on enqueue. For each dequeued cell, generate in-bounds open neighbors whose distance is still -1 .

For the grid below, from top-left to bottom-right, `#` blocks movement:

TEXT

```
...
.#.
...
```

One shortest route moves right, right, down, down for four steps. Time and memory are $O(\text{rows} * \text{cols})$. If moves have weights 0 and 1, use 0-1 BFS with a deque; for arbitrary nonnegative costs, use Dijkstra.

Pattern 5: undirected cycle detection

During DFS of an undirected graph, an edge back to the immediate parent is the same tree edge viewed in reverse and is not a cycle. An edge to any other discovered vertex proves a cycle.

Carry `(vertex, parent)` state. Invariant: the DFS tree edges connect each discovered nonroot vertex to its parent. Encountering a previously discovered neighbor different from that parent closes a path through the DFS tree.

Parallel undirected edges complicate a parent-by-vertex check: the second parallel edge to the parent can itself form a length-two multi-edge cycle. If multigraphs are allowed, assign edge IDs and skip only the exact incoming edge ID. The runnable helper assumes a simple symmetric graph.

Pattern 6: directed cycles and topological order

Directed DFS uses three colors:

- white: undiscovered;
- gray: active on the current recursion path;
- black: fully processed.

An edge to gray is a back edge and proves a directed cycle. An edge to black does not, because that vertex's path has already finished.

Kahn's topological algorithm gives an iterative alternative. Compute indegrees, enqueue every zero-indegree vertex, repeatedly emit one and decrement its outgoing neighbors. Invariant: every queued vertex has no incoming edge from the remaining graph, so placing it next respects all dependencies. If fewer than V vertices are emitted, the remaining subgraph has no zero-indegree vertex and contains a cycle.

A topological order exists only for a directed acyclic graph and may not be unique. Use a priority queue instead of a FIFO queue if the smallest deterministic vertex should be chosen at each step, increasing frontier operations to $O(\log V)$.

Pattern 7: bipartite testing

A graph is bipartite when vertices can be colored with two colors so every edge crosses colors. BFS or DFS each component, assigning an uncolored neighbor the opposite color. An edge whose endpoints have the same color proves failure.

Invariant: every processed edge among colored vertices crosses colors. Because disconnected graphs can contain the violating component, scan every vertex, not only one start.

Equivalently, an undirected graph is bipartite exactly when it has no odd-length cycle. A self-loop fails immediately. Time is $O(V + E)$, space $O(V)$.

Shortest-path decision table

Edge model	Algorithm	Typical time
unweighted/unit weight	BFS	$O(V + E)$
weights only 0 or 1	0-1 BFS	$O(V + E)$
nonnegative weights	Dijkstra with binary heap	$O((V + E) \log V)$
DAG with any weights	topological relaxation	$O(V + E)$
negative edges, detect reachable negative cycle	Bellman-Ford	$O(VE)$
all-pairs, dense and moderate V	Floyd-Warshall	$O(V^3)$ time, $O(V^2)$ space

A negative cycle reachable from the source makes shortest distance undefined for vertices reachable from that cycle: the path cost can decrease without bound. Bellman-Ford relaxes every edge $V - 1$ times, then a further successful relaxation detects such a cycle. Early termination is safe when one full pass makes no change.

Floyd-Warshall dynamic programming allows intermediate vertices one by one:

TEXT

```
dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])
```

At phase k , the invariant is that distances use only intermediate vertices from the processed set. Guard unreachable sentinels before addition and choose a width that cannot overflow.

Pattern 8: Dijkstra with path reconstruction

Dijkstra maintains tentative distances and a min-priority queue. Repeatedly settle the reachable unsettled vertex with smallest distance, then relax its outgoing edges.

For edge $u \rightarrow v$ of weight w , relaxation tests whether $\text{dist}[u] + w < \text{dist}[v]$. If so, update distance, set $\text{parent}[v] = u$, and add a new queue state. Java's `PriorityQueue` has no efficient decrease-key, so old states remain. When polled, discard a state whose stored distance differs from the current $\text{dist}[\text{vertex}]$.

Correctness invariant: every settled vertex has its final shortest distance. Nonnegative weights are essential: any path reaching an unsettled vertex through another unsettled vertex cannot later reduce the current minimum by adding a negative amount.

TRACE IT | Dry run

Edges: $0 \rightarrow 1$ (4), $0 \rightarrow 2$ (1), $2 \rightarrow 1$ (2), $1 \rightarrow 3$ (1), $2 \rightarrow 3$ (5). Start 0.

- 1 Settle 0; tentative distances: $1=4$, $2=1$.
- 2 Settle 2; improve 1 to 3 through 2, set 3 to 6.
- 3 Settle 1 at 3; improve 3 to 4.
- 4 Settle 3 at 4. Old states for 1 at 4 and 3 at 6 are stale.

Reconstruct target 3 by parents $3 \leftarrow 1 \leftarrow 2 \leftarrow 0$, reverse to $[0, 2, 1, 3]$. If unreachable, return an explicit empty path with an infinite-distance sentinel rather than fabricating a route.

Minimum spanning tree versus shortest paths

For a connected undirected weighted graph, a minimum spanning tree (MST) connects every vertex with minimum total edge weight. It does not generally preserve shortest paths from any source.

Kruskal and DSU

Sort edges by increasing weight. Add an edge only when its endpoints currently belong to different components. A disjoint-set union (DSU) supports **find** with path compression and **union** by size/rank.

Cut property: for any cut of the vertices, a lightest edge crossing that cut is safe for some MST. Kruskal's next edge connects two current components and is a lightest remaining edge across their cut; adding it cannot create a cycle and is safe. Stop after $V - 1$ accepted edges.

Time is $O(E \log E)$ for sorting; DSU operations are nearly constant amortized, formally $O(\alpha(V))$. If fewer than $V - 1$ edges are accepted, the graph is disconnected. Return a minimum spanning forest only if that is the stated contract.

Prim

Prim grows one tree from a start vertex. A min-heap stores edges crossing from the included set to excluded vertices. Repeatedly choose the lightest crossing edge, skip it if its destination is already included, and add the new vertex's outgoing edges. The same cut property proves safety.

With adjacency lists and a binary heap, time is $O(E \log V)$. Prim is natural when the graph is already in adjacency form; Kruskal is natural for an edge list and gives explicit component merging.

Negative edge weights are allowed in MST algorithms. The nonnegative restriction belongs to Dijkstra, not to spanning trees.

Advanced decomposition boundary

Some SDE-2 interviews ask recognition and high-level mechanics rather than full code for these families.

- **Strongly connected components (SCC):** in a directed graph, every pair inside an SCC reaches each other. Kosaraju uses finishing order plus a reversed graph; Tarjan uses one DFS with discovery indexes, low links, and a stack. Condensing SCCs produces a DAG.
- **Bridges and articulation points:** in an undirected graph, a bridge removal increases component count; an articulation vertex removal does likewise. DFS low-link values track the earliest discovery reachable through tree edges plus at most one back edge. A tree edge (u, v) is a bridge when $\text{low}[v] > \text{discovery}[u]$ in a simple graph.
- **Euler paths:** depend on vertex degrees and connectedness of nonzero-degree vertices, not on MST or shortest paths.
- **Network flow:** capacities and conservation require augmenting-path or blocking-flow algorithms, not greedy shortest paths alone.

Know the recognition signal, invariant vocabulary, complexity, and implementation hazards. Do not force every advanced graph problem into BFS, Dijkstra, or DSU.

Runnable Java 21 reference implementation

Run with `java -ea GraphInterviewPatterns`.

JAVA (continued 1/13)

```
import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;
import java.util.PriorityQueue;

public final class GraphInterviewPatterns {
    private GraphInterviewPatterns() {}

    public record WeightedEdge(int to, int weight) {}

    public record UndirectedEdge(int first, int second, int weight) {}

    public record PathResult(long distance, List<Integer> path) {
        public boolean reachable() {
            return distance != Long.MAX_VALUE;
        }
    }

    private record NodeDistance(int vertex, long distance) {}

    public static List<Integer> bfsOrder(int[][] adjacency, int start) {
        validateAdjacency(adjacency);
        validateVertex(start, adjacency.length);
        boolean[] seen = new boolean[adjacency.length];
        ArrayDeque<Integer> queue = new ArrayDeque<>();
        List<Integer> order = new ArrayList<>();
        seen[start] = true;
```

JAVA (continued 2/13)

```
        queue.addLast(start);
        while (!queue.isEmpty()) {
            int vertex = queue.removeFirst();
            order.add(vertex);
            for (int neighbor : adjacency[vertex]) {
                if (!seen[neighbor]) {
                    seen[neighbor] = true;
                    queue.addLast(neighbor);
                }
            }
        }
        return List.copyOf(order);
    }

    public static List<Integer> dfsOrder(int[][] adjacency, int start) {
        validateAdjacency(adjacency);
        validateVertex(start, adjacency.length);
        boolean[] seen = new boolean[adjacency.length];
        ArrayDeque<Integer> stack = new ArrayDeque<>();
        List<Integer> order = new ArrayList<>();
        seen[start] = true;
        stack.push(start);
        while (!stack.isEmpty()) {
            int vertex = stack.pop();
            order.add(vertex);
            int[] neighbors = adjacency[vertex];
            for (int i = neighbors.length - 1; i >= 0; i--) {
                int neighbor = neighbors[i];
                if (!seen[neighbor]) {
                    seen[neighbor] = true;
                    stack.push(neighbor);
                }
            }
        }
    }
}
```


JAVA (continued 3/13)

```
        return List.copyOf(order);
    }

    public static int countUndirectedComponents(int[][] adjacency) {
        validateAdjacency(adjacency);
        boolean[] seen = new boolean[adjacency.length];
        int components = 0;
        for (int start = 0; start < adjacency.length; start++) {
            if (seen[start]) {
                continue;
            }
            components++;
            ArrayDeque<Integer> stack = new ArrayDeque<>();
            seen[start] = true;
            stack.push(start);
            while (!stack.isEmpty()) {
                int vertex = stack.pop();
                for (int neighbor : adjacency[vertex]) {
                    if (!seen[neighbor]) {
                        seen[neighbor] = true;
                        stack.push(neighbor);
                    }
                }
            }
        }
        return components;
    }

    public static int shortestGridSteps(int[][] grid, int startRow, int startCol,
        int targetRow, int targetCol) {
        int cols = validateRectangularGrid(grid);
        validateCell(startRow, startCol, grid.length, cols);
        validateCell(targetRow, targetCol, grid.length, cols);
        if (grid[startRow][startCol] != 0 || grid[targetRow][targetCol] != 0) {
```

JAVA (continued 4/13)

```

        return -1;
    }
    int[][] distance = new int[grid.length][cols];
    for (int[] row : distance) {
        Arrays.fill(row, -1);
    }
    int[] rowDelta = {-1, 1, 0, 0};
    int[] colDelta = {0, 0, -1, 1};
    ArrayDeque<Integer> queue = new ArrayDeque<>();
    distance[startRow][startCol] = 0;
    queue.addLast(startRow * cols + startCol);
    while (!queue.isEmpty()) {
        int cell = queue.removeFirst();
        int row = cell / cols;
        int col = cell % cols;
        if (row == targetRow && col == targetCol) {
            return distance[row][col];
        }
        for (int direction = 0; direction < 4; direction++) {
            int nextRow = row + rowDelta[direction];
            int nextCol = col + colDelta[direction];
            if (nextRow >= 0 && nextRow < grid.length
                && nextCol >= 0 && nextCol < cols
                && grid[nextRow][nextCol] == 0
                && distance[nextRow][nextCol] == -1) {
                distance[nextRow][nextCol] = distance[row][col] + 1;
                queue.addLast(nextRow * cols + nextCol);
            }
        }
    }
    return -1;
}

public static boolean hasUndirectedCycle(int[][] adjacency) {

```

JAVA (continued 5/13)

```
        validateAdjacency(adjacency);
        boolean[] seen = new boolean[adjacency.length];
        for (int vertex = 0; vertex < adjacency.length; vertex++) {
            if (!seen[vertex] && undirectedCycleDfs(adjacency, vertex, -1, seen)) {
                return true;
            }
        }
        return false;
    }

    public static boolean hasDirectedCycle(int[][] adjacency) {
        validateAdjacency(adjacency);
        try {
            topologicalOrder(adjacency);
            return false;
        } catch (IllegalArgumentException cycle) {
            return true;
        }
    }

    public static List<Integer> topologicalOrder(int[][] adjacency) {
        validateAdjacency(adjacency);
        int[] indegree = new int[adjacency.length];
        for (int[] neighbors : adjacency) {
            for (int neighbor : neighbors) {
                indegree[neighbor]++;
            }
        }
        ArrayDeque<Integer> queue = new ArrayDeque<>();
        for (int vertex = 0; vertex < indegree.length; vertex++) {
            if (indegree[vertex] == 0) {
                queue.addLast(vertex);
            }
        }
    }
```

JAVA (continued 6/13)

```
List<Integer> order = new ArrayList<>();
while (!queue.isEmpty()) {
    int vertex = queue.removeFirst();
    order.add(vertex);
    for (int neighbor : adjacency[vertex]) {
        if (--indegree[neighbor] == 0) {
            queue.addLast(neighbor);
        }
    }
}
if (order.size() != adjacency.length) {
    throw new IllegalArgumentException("directed graph contains a cycle");
}
return List.copyOf(order);
}

public static boolean isBipartite(int[][] adjacency) {
    validateAdjacency(adjacency);
    int[] color = new int[adjacency.length];
    Arrays.fill(color, -1);
    for (int start = 0; start < adjacency.length; start++) {
        if (color[start] != -1) {
            continue;
        }
        color[start] = 0;
        ArrayDeque<Integer> queue = new ArrayDeque<>();
        queue.addLast(start);
        while (!queue.isEmpty()) {
            int vertex = queue.removeFirst();
            for (int neighbor : adjacency[vertex]) {
                if (color[neighbor] == -1) {
                    color[neighbor] = 1 - color[vertex];
                    queue.addLast(neighbor);
                } else if (color[neighbor] == color[vertex]) {

```

JAVA (continued 7/13)

```

        return false;
    }
}
}
return true;
}

public static PathResult dijkstra(List<List<WeightedEdge>> graph,
    int source, int target) {
    validateWeightedGraph(graph);
    validateVertex(source, graph.size());
    validateVertex(target, graph.size());
    long[] distance = new long[graph.size()];
    int[] parent = new int[graph.size()];
    Arrays.fill(distance, Long.MAX_VALUE);
    Arrays.fill(parent, -1);
    PriorityQueue<NodeDistance> queue = new PriorityQueue<>(
        Comparator.comparingLong(NodeDistance::distance)
            .thenComparingInt(NodeDistance::vertex));
    distance[source] = 0;
    queue.add(new NodeDistance(source, 0));
    while (!queue.isEmpty()) {
        NodeDistance current = queue.remove();
        int vertex = current.vertex();
        if (current.distance() != distance[vertex]) {
            continue;
        }
        if (vertex == target) {
            break;
        }
        for (WeightedEdge edge : graph.get(vertex)) {
            long candidate = Math.addExact(distance[vertex], edge.weight());
            if (candidate < distance[edge.to()]) {

```

JAVA (continued 8/13)

```

        distance[edge.to()] = candidate;
        parent[edge.to()] = vertex;
        queue.add(new NodeDistance(edge.to(), candidate));
    }
}
}
if (distance[target] == Long.MAX_VALUE) {
    return new PathResult(Long.MAX_VALUE, List.of());
}
List<Integer> reversed = new ArrayList<>();
for (int vertex = target; vertex != -1; vertex = parent[vertex]) {
    reversed.add(vertex);
}
Collections.reverse(reversed);
return new PathResult(distance[target], List.copyOf(reversed));
}

public static long kruskalMstWeight(int vertices, List<UndirectedEdge> edges) {
    if (vertices < 0 || edges == null) {
        throw new IllegalArgumentException("invalid vertices or edges");
    }
    List<UndirectedEdge> sorted = new ArrayList<>(edges);
    for (UndirectedEdge edge : sorted) {
        if (edge == null) {
            throw new IllegalArgumentException("edge must not be null");
        }
        validateVertex(edge.first(), vertices);
        validateVertex(edge.second(), vertices);
    }
    sorted.sort(Comparator.comparingInt(UndirectedEdge::weight)
        .thenComparingInt(UndirectedEdge::first)
        .thenComparingInt(UndirectedEdge::second));
    DisjointSet sets = new DisjointSet(vertices);
    int accepted = 0;

```


JAVA (continued 9/13)

```
        long weight = 0;
        for (UndirectedEdge edge : sorted) {
            if (sets.union(edge.first(), edge.second())) {
                weight = Math.addExact(weight, edge.weight());
                accepted++;
                if (accepted == vertices - 1) {
                    break;
                }
            }
        }
        if (vertices > 0 && accepted != vertices - 1) {
            throw new IllegalArgumentException("graph is disconnected");
        }
        return weight;
    }

    private static boolean undirectedCycleDfs(int[][] graph, int vertex,
        int parent, boolean[] seen) {
        seen[vertex] = true;
        for (int neighbor : graph[vertex]) {
            if (!seen[neighbor]) {
                if (undirectedCycleDfs(graph, neighbor, vertex, seen)) {
                    return true;
                }
            } else if (neighbor != parent) {
                return true;
            }
        }
        return false;
    }

    private static final class DisjointSet {
        private final int[] parent;
        private final int[] size;
```

JAVA (continued 10/13)

```
private DisjointSet(int count) {
    parent = new int[count];
    size = new int[count];
    for (int i = 0; i < count; i++) {
        parent[i] = i;
        size[i] = 1;
    }
}

private int find(int value) {
    int root = value;
    while (root != parent[root]) {
        root = parent[root];
    }
    while (value != root) {
        int next = parent[value];
        parent[value] = root;
        value = next;
    }
    return root;
}

private boolean union(int first, int second) {
    int rootA = find(first);
    int rootB = find(second);
    if (rootA == rootB) {
        return false;
    }
    if (size[rootA] < size[rootB]) {
        int temporary = rootA;
        rootA = rootB;
        rootB = temporary;
    }
}
```

JAVA (continued 11/13)

```
        }
        parent[rootB] = rootA;
        size[rootA] += size[rootB];
        return true;
    }
}

private static void validateAdjacency(int[][] adjacency) {
    if (adjacency == null) {
        throw new IllegalArgumentException("adjacency must not be null");
    }
    for (int[] neighbors : adjacency) {
        if (neighbors == null) {
            throw new IllegalArgumentException("neighbor list must not be null");
        }
        for (int neighbor : neighbors) {
            validateVertex(neighbor, adjacency.length);
        }
    }
}

private static void validateWeightedGraph(List<List<WeightedEdge>> graph) {
    if (graph == null) {
        throw new IllegalArgumentException("graph must not be null");
    }
    for (List<WeightedEdge> edges : graph) {
        if (edges == null) {
            throw new IllegalArgumentException("edge list must not be null");
        }
        for (WeightedEdge edge : edges) {
            if (edge == null || edge.weight() < 0) {
                throw new IllegalArgumentException("Dijkstra needs nonnegative edges");
            }
        }
    }
}
```

JAVA (continued 12/13)

```
        validateVertex(edge.to(), graph.size());
    }
}

private static int validateRectangularGrid(int[][] grid) {
    if (grid == null || grid.length == 0 || grid[0] == null
        || grid[0].length == 0) {
        throw new IllegalArgumentException("nonempty grid required");
    }
    int cols = grid[0].length;
    for (int[] row : grid) {
        if (row == null || row.length != cols) {
            throw new IllegalArgumentException("grid must be rectangular");
        }
    }
    return cols;
}

private static void validateCell(int row, int col, int rows, int cols) {
    if (row < 0 || row >= rows || col < 0 || col >= cols) {
        throw new IllegalArgumentException("cell outside grid");
    }
}

private static void validateVertex(int vertex, int count) {
    if (vertex < 0 || vertex >= count) {
        throw new IllegalArgumentException("vertex outside graph");
    }
}

public static void main(String[] args) {
    int[][] undirected = {{1, 2}, {0, 3}, {0, 3}, {1, 2, 4}, {3}, {}};
}
```

JAVA (continued 13/13)

```

    assert bfsOrder(undirected, 0).equals(List.of(0, 1, 2, 3, 4));
    assert dfsOrder(undirected, 0).equals(List.of(0, 1, 3, 4, 2));
    assert countUndirectedComponents(undirected) == 2;
    assert hasUndirectedCycle(undirected);
    assert isBipartite(undirected);

    int[][] grid = {{0, 0, 0}, {0, 1, 0}, {0, 0, 0}};
    assert shortestGridSteps(grid, 0, 0, 2, 2) == 4;

    int[][] dag = {{1, 2}, {3}, {3}, {}};
    List<Integer> topo = topologicalOrder(dag);
    assert topo.indexOf(0) < topo.indexOf(1);
    assert topo.indexOf(0) < topo.indexOf(2);
    assert topo.indexOf(1) < topo.indexOf(3);
    assert !hasDirectedCycle(dag);
    assert hasDirectedCycle(new int[][] {{1}, {2}, {0}});

    List<List<WeightedEdge>> weighted = List.of(
        List.of(new WeightedEdge(1, 4), new WeightedEdge(2, 1)),
        List.of(new WeightedEdge(3, 1)),
        List.of(new WeightedEdge(1, 2), new WeightedEdge(3, 5)),
        List.of());
    PathResult shortest = dijkstra(weighted, 0, 3);
    assert shortest.distance() == 4;
    assert shortest.path().equals(List.of(0, 2, 1, 3));

    List<UndirectedEdge> edges = List.of(
        new UndirectedEdge(0, 1, 1), new UndirectedEdge(1, 2, 2),
        new UndirectedEdge(0, 2, 4), new UndirectedEdge(2, 3, 1),
        new UndirectedEdge(1, 3, 5));
    assert kruskalMstWeight(4, edges) == 4;
}

```

Complexity and edge-case table

Family	Time	Space	Critical precondition
BFS/DFS/components	$O(V + E)$	$O(V)$	valid adjacency and discovery marking
grid BFS	$O(\text{rows} * \text{cols})$	$O(\text{rows} * \text{cols})$	unit-cost legal moves
cycle/bipartite/topological	$O(V + E)$	$O(V)$	direction/simple-graph policy
Dijkstra binary heap	$O((V + E) \log V)$	$O(V + E)$ with stale states	nonnegative weights
Bellman-Ford	$O(VE)$	$O(V)$	edge list and overflow-safe sentinel
Floyd-Warshall	$O(V^3)$	$O(V^2)$	moderate dense graph
Kruskal	$O(E \log E)$	$O(V + E)$	undirected graph; connected for MST
Prim binary heap	$O(E \log V)$	$O(V + E)$	undirected adjacency

WATCH OUT | Edge cases and common mistakes

- 1 **Direction lost.** Adding reverse edges changes reachability and cycle semantics.
- 2 **Visited marked on dequeue.** Converging paths enqueue duplicates; mark on discovery.
- 3 **Only one component scanned.** Cycle and bipartite checks must cover disconnected graphs.
- 4 **Recursive stack overflow.** Use iterative traversal for deep untrusted graphs.
- 5 **Undirected parent edge called a cycle.** Skip the incoming tree edge, with edge IDs for multigraphs.
- 6 **Directed seen state collapsed to boolean.** Cycle detection needs active versus finished state, or Kahn's processed count.
- 7 **BFS on weighted edges.** It minimizes edge count, not arbitrary total cost.
- 8 **Dijkstra with negative edge.** The settlement proof fails even without a negative cycle.
- 9 **Stale heap entry processed.** Compare queued distance with current best.
- 10 **No parent update.** A distance alone cannot reconstruct the requested path.
- 11 **Infinity arithmetic overflow.** Never add an edge to an unreachable sentinel.
- 12 **MST confused with shortest-path tree.** Objectives differ.
- 13 **Disconnected MST silently returned.** Define forest versus rejection.
- 14 **External IDs used as array indexes.** Compact and validate mapping.
- 15 **Output size ignored.** All-pairs paths or all simple paths can dominate computation and memory.

SDE-2 scale and production follow-ups

- **Graph ingestion:** validate endpoints, direction, duplicates, and weight units. Bad edges should not become array exceptions deep in an algorithm.
- **Memory layout:** object-per-edge adjacency is convenient but expensive. Large graphs may use compressed sparse row arrays or off-heap storage.
- **Determinism:** neighbor order changes traversal and equally optimal path output. Sort or define tie-breaking if consumers need stable results.
- **Dynamic graphs:** cached paths and components become stale after updates. Recompute, incrementally maintain, or serve versioned snapshots.
- **Parallel edges:** retain the cheapest for shortest path only when other edge identity/policy does not matter.
- **Huge frontiers:** BFS can exhaust memory before time. Bidirectional search, external-memory traversal, or domain heuristics may help.
- **Distributed boundary:** a graph spread across machines makes random neighbor access and global priority queues expensive. Partitioning, messaging, consistency, and failure dominate textbook complexity.
- **Rate limits:** route planning over external services needs batched lookups, caching, retry policy, and partial failure handling.
- **Observability:** measure vertices/edges explored, frontier peak, relaxations, stale queue entries, and disconnected/unreachable rates.
- **Security:** cap graph size and path output; adversarial graphs can maximize frontier, recursion depth, or collision/cardinality costs.
- **Versioned weights:** time-dependent travel costs can violate static edge assumptions and even the FIFO property required by specialized routing algorithms.

TRY IT | Exercises with model checkpoints

Exercise 1: shortest unweighted path reconstruction

Return vertex path from source to target with BFS.

Model checkpoints: parent assigned on first discovery; stop when target is discovered or dequeued under a documented rule; unreachable returns empty; reverse parent chain; $O(V + E)$.

Exercise 2: course schedule diagnosis

Return either a valid course order or one directed cycle.

Model checkpoints: Kahn returns order but not a cycle directly; three-color DFS plus parent links can reconstruct a back-edge cycle; distinguish prerequisite edge direction; include disconnected courses.

Exercise 3: 0-1 BFS

Find shortest costs when every edge weight is zero or one.

Model checkpoints: relax into an `ArrayDeque`; zero-weight improvement goes to front, one-weight to back; stale distance checks still matter; time $O(V + E)$; reject other weights.

Exercise 4: Bellman-Ford paths

Return distances and mark vertices affected by a reachable negative cycle.

Model checkpoints: relax only from reachable vertices; after $V-1$ passes, collect vertices improved on pass V ; traverse forward from them to mark affected outputs; parent chains through negative cycles are not ordinary shortest paths.

Exercise 5: Prim MST edges

Return the actual MST from weighted adjacency.

Model checkpoints: heap state includes weight, from, to; skip destinations already in tree; start a new tree only for a forest contract; accepted edges total $V-1$; negative weights are allowed.

Exercise 6: accounts merge

Group records sharing identifiers.

Model checkpoints: model identifiers as vertices or union record indexes via shared identifiers; DSU supports incremental unions; map external strings to indexes; deterministic output requires sorted members and group order.

Exercise 7: bridge recognition

Find critical undirected connections.

Model checkpoints: discovery time and low link; skip incoming edge ID, not every edge to parent; (u, v) is a bridge when $low[v] > disc[u]$; scan components; recursive depth hazard.

Interview answer checklist

- ☐ I defined vertices, edges, direction, weight meaning, and multiplicity.
- ☐ I chose representation from density and operations.
- ☐ I mark discovered vertices when scheduling them.
- ☐ I scan disconnected components when required.
- ☐ I chose shortest-path algorithm from the weight domain.
- ☐ I stated Dijkstra's nonnegative-edge invariant and stale-entry rule.
- ☐ I reconstruct paths only from updated parent state.
- ☐ I distinguish MST, shortest path, connectivity, and ordering objectives.
- ☐ I handle numeric sentinels, recursion depth, and result determinism.
- ☐ I can identify SCC, bridges, flow, and external-scale boundaries.

RECAP | Summary

Graph correctness begins with modeling. Adjacency lists make sparse traversal linear; BFS discovers minimum edge counts; DFS exposes component and dependency structure; grid BFS generates neighbors implicitly. Parent state distinguishes undirected cycles, active state or indegree detects directed cycles, and two-color propagation tests bipartiteness. Weight constraints select BFS, Dijkstra, DAG relaxation, Bellman-Ford, or Floyd-Warshall. Dijkstra settles nonnegative distances and reconstructs paths; Kruskal and Prim use cut safety to build minimum spanning trees. SCCs and bridges mark the next decomposition boundary. The SDE-2 answer connects those invariants to scale, layout, deterministic output, validation, and operational constraints.

Part II - Practice and Handoff

Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: traversal and components
- Interview Core: cycle detection, topological sort, and shortest paths
- SDE-2 Follow-up: path reconstruction and scale
- Challenge: adapt algorithms to grids or evolving connectivity

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

[Previous book: Study Step 14 - Heaps, Priority Queues, Selection, and Top-K](#)

[Series index](#)

[Next book: Study Step 16 - Greedy Algorithms: Recognition, Proofs, and Scheduling](#)

TRY IT | Completion check

- ☐ I can recognize the core patterns without being told the data structure or algorithm.
- ☐ I can implement the representative Java templates from memory.
- ☐ I can explain why each implementation is correct.
- ☐ I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- ☐ I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Complete the study steps in order when rebuilding from fundamentals, or enter at the first step whose readiness checks expose a gap. The same public study codes appear on the website and every PDF cover. Click a title when your PDF viewer permits local sibling-file links.

STEP	FOCUSED LEARNING PATH
01	Java Foundations for Problem Solving
02	Time and Space Complexity for Java Interviews
03A	Number Systems and Math Foundations for DSA Interviews
03B	Number Systems Interview Patterns and Rapid Revision
04	Bit Manipulation in Java
05	Loop Mastery, Patterns, and Index Calculations
06	Arrays and Array Problem-Solving Patterns
07	Strings and String Problem-Solving Patterns
08	Hashing: Maps, Sets, Frequency, and Prefix State
09	Recursion and Backtracking in Java
10	Linked Lists: Pointer Reasoning and Mutation
11	Stacks, Queues, Deques, and Monotonic Patterns
12	Binary Search: Bounds, Answers, and Invariants
13	Trees, Binary Search Trees, and Tries
14	Heaps, Priority Queues, Selection, and Top-K
15	Graphs: Traversal, Ordering, Paths, and Union-Find
16	Greedy Algorithms: Recognition, Proofs, and Scheduling
17	Dynamic Programming: State, Transitions, and Optimization
18A	Advanced Java A: JVM and Execution
18B	Advanced Java B: Language, OOP, and Modern Java
18C	Advanced Java C: Collections, Streams, and I/O
18D	Advanced Java D: Concurrency and the Memory Model
18E	Advanced Java E: Performance, Diagnostics, and GC Incidents
18F	Advanced Java F: Design, Backend, Testing, and Security
18G	Advanced Java G: Question Bank, Study Plan, and Reference
18H	Advanced Java H: Spring Boot and REST APIs
18I	Advanced Java I: Persistence, SQL, JPA, and Caching
18J	Advanced Java J: Distributed Systems and System Design

Study Step 03 uses linked foundations (03A) and workbook (03B) books. Study Step 18 uses ten focused books (18A-18J), including a three-book backend specialist track. These suffixes keep every file focused while preserving one unambiguous route.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer, the creator and founding author of the Java SDE-2 Interview Preparation Series, and its Editor-in-Chief and Chief Auditor. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial leadership

As Editor-in-Chief, Vinay owns the learning sequence, scope, voice, and publication decisions. As Chief Auditor, he owns the Java accuracy standard, validation evidence, attribution review, and release-readiness gate. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: [LinkedIn](#) | [GitHub](#)

Copyright and Publishing Notes

Copyright 2026 Vinay Reddy Kalluri and credited contributors. Open educational edition.

Book prose, exercises, diagrams, and PDFs are licensed under Creative Commons Attribution 4.0 International (CC BY 4.0). Build scripts and source code are licensed under MIT. Individual credit is recorded in AUTHORS.md and Git history.

Repository: <https://github.com/vinayreddykalluri/SDE2-Interview-Handbook>. Attribution does not imply endorsement. Java and related marks are owned by their respective holders.

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Study Step 15 of 18. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.